MILESTONE: Beta
TEAM NAME: Team Gathering
GAME NAME: Forest of the Food
NAME: Minjae Kyung
JOBS: Technical Director
DEGREE PROGRAM: RTIS
GAME DESCRIPTION: The forest of food is player finds,steals and collects ingredient of foods and makes favorite seasonal food. And player should satisfy fairy of the forest to escape the forest of food.
LINKS: https://bitbucket.org/llttime/llttime/src/master/

Work Done

I made y-coordinate base drawing function in GameObject Manager class. Now (In beta) it is only applied in trees and player. but I'll apply to all object

```
void GameObjectManager::DrawAll(TransformMatrix& cameraMatrix, unsigned int season)
{
    itemObjects.Draw(cameraMatrix, season);
    animalObjects.Draw(cameraMatrix);

    auto treeAfterPlayer = std::find_if(begin(treeObjects.trees), end(treeObjects.trees),
        [this](Environment* elem)
        {
            return this->player.GetPosition().y > elem->GetPosition().y;
        });
    for_each(begin(treeObjects.trees), treeAfterPlayer,
        [cameraMatrix](Environment* elem)
        {
            elem->Draw(cameraMatrix);
        });
    player.Draw(cameraMatrix);
    for_each(treeAfterPlayer, end(treeObjects.trees),
        [cameraMatrix](Environment* elem)
        {
            elem->Draw(cameraMatrix);
        });
    player.DrawHUD();
}
```

I made SFX and BGM to create sound in game

```cpp
#pragma once
#include <SFML/Audio.hpp>
#include <doodle/doodle.hpp>

class SFX
{
    sf::Sound sound;
public:
    SFX(const std::string& filePath, bool soundRelative = false);
    void Play();
    void PlayOnly();
    bool isRelative();
};

class BGM
{
    sf::Music* music;

public:
    BGM(const std::string& filePath, bool loop = true);

    void Play();
    void Stop();
    bool isLoop();
};
```

I made soundManager to manage SFX and BGM

```cpp
class SoundManager
{
    class SFXManager
    {
        std::map<std::string, sf::SoundBuffer*> pathToBuffer
    public:
        sf::SoundBuffer* Load(const std::string& filePath);
        void Unload();
    };

    class BGMManager
    {
        std::map<std::string, sf::Music*> pathToMusic;

    public:
        sf::Music* Load(const std::string& filePath);
        void Unload();
    };

    SFXManager sfxManager;
    BGMManager bgmManager;

public:
    sf::SoundBuffer* LoadSFX(const std::string& filePath);
    sf::Music* LoadBGM(const std::string& filePath);

    void Unload();

};
```

In addition, I created map of SFXs in Object class

```
void Object::AddSFX(std::string nameSFX, std::string filePath)
{
    soundEffects.insert({ nameSFX, new SFX{filePath} });
}
void Object::PlaySFX(std::string nameSFX)
{
    soundEffects[nameSFX]->Play();
}

void Object::Unload()
{
    for (std::pair<std::string, SFX*> sfx : soundEffects)
    {
        delete sfx.second;
    }
    soundEffects.clear();
}
```

And I made unpolished version of knife

```
void Knife::Update(GameObjectManager& objManager, double dt)
{
    position = objManager.player.GetPosition() + objManager.player.GetHotSpot(1);

    sprite.Update(dt);

    if (durationTimer > 0)
    {
        Direction temp;
        for(int i = 0; i< objManager.animalObjects.GetAnimals().size();i++)
        {
            Creature* copy = objManager.animalObjects.GetAnimals()[i];
            temp = CollisionCheck(copy, dt);
            std::cout << temp;

            if (temp != Direction::notD)
            {
                std::cout << "Direction : " << temp << std::endl;
                std::cout << "[Animal] Wn";
                copy->CollisionInteractWithKnife(objManager, dt);
            }
        }
        durationTimer -= dt;
    }
    coolTimer -= dt;
```

```
    objectMatrix = TranslateMatrix(position);
    if (isFlipped == true)
    {
        objectMatrix *= ScaleMatrix({ -1.0, 1.0 });
    }
    else if (isFlipped == false)
    {
        objectMatrix *= ScaleMatrix({ 1.0,1.0 });
    }


    HitboxUpdate();
}

void Knife::useKnife()
{
    if (coolTimer <= 0)
    {
        durationTimer = duration;
        coolTimer = coolTime;
        //PlaySFX("Attack");
        sprite.PlayAnimation(1);
    }
}
```